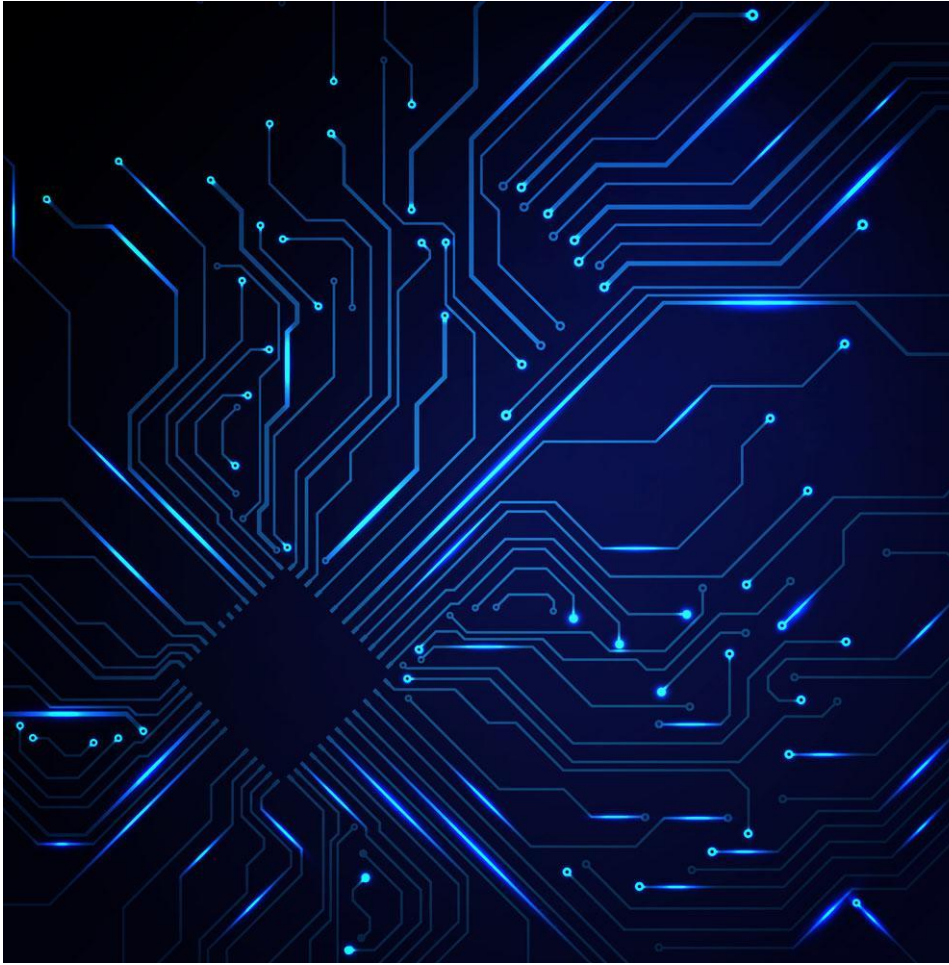




注意

1. 繳交時一律轉 PDF 檔
2. 繳交期限為
隔週三上午九點
3. 一人繳交一份
4. 檔名：學號_HW?.pdf
檔名請按照作業檔名格式進行填寫
未依照格式不予批改

2026/05/05

實驗十一

姓名：陳篁瑋

學號：01257024

班級：資工 3A

E-mail：henry246813579@gmail.com



1、 RS-232 Reciever (一)

■ 實驗說明：

1. 寫出 RS-232 Reciever，並顯示其模擬圖。

2. 輸入：

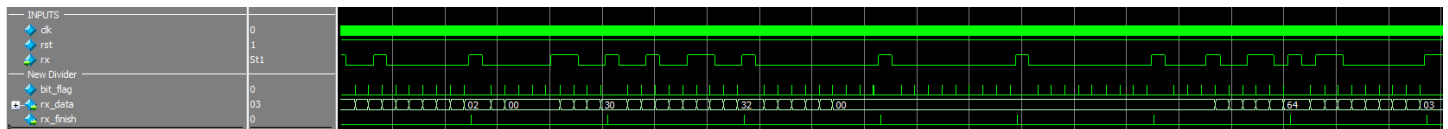
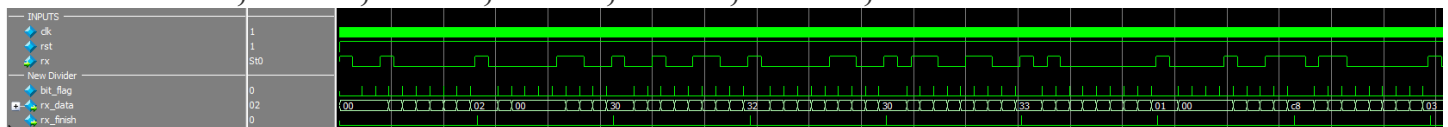
- clk : 50MHz
- rst
- rx : 接收到資料(1-bit)

3. 輸出：

- rx_data[7:0] : 將收到的資料解析、組合然後輸出。
- rx_finish : 標示完整接收到 1 個 byte 的 trigger。

4. 測資：

- 8'h02, 8'h30, 8'h32, 8'h30, 8'h33, 8'h01, 8'h03
- 8'h02, 8'h30, 8'h32, 8'h00, 8'h00, 8'h00, 8'h03



■ 系統架構程式碼與程式碼說明

截圖請善用 win+shift+S

RS232_rx

```
1 module rs232_rx(
2     input logic clk,
3     input logic rst,
4     input logic tx_ack,
5     input logic rx,
6     output logic [7:0]addr,
7     output logic [7:0]data,
8     output logic write,
9     output logic tx_req,
10    output logic [1:0]tx_cnt
11);
12
13
14    //bit_counter
15    logic rst_bit_cnt;
16    logic bit_flag;
17    logic [5:0]bit_cnt;
18
19    always_ff@(posedge clk)
20    begin
21        if(~rst || rst_bit_cnt)
22            bit_cnt <= 0;
23        else if(bit_flag)
24            bit_cnt <= bit_cnt + 1'b1;
25    end
26
27
28    //baud_counter
29    logic rst_baud_cnt;
30    logic [15:0]baud_cnt;
31
32    always_ff@(posedge clk)
33    begin
34        if(~rst || rst_baud_cnt)
35            baud_cnt <= 0;
36        else
37            baud_cnt <= baud_cnt + 1'b1;
38    end
39
40    //pkg_counter
41    logic [2:0] pkg_cnt;
42    logic rst_pkg_cnt;
43    logic rx_finish;
44
45    always_ff@(posedge clk)
46    begin
47        if(~rst || rst_pkg_cnt)
48            pkg_cnt <= 0;
49        else if(rx_finish)
50            pkg_cnt <= pkg_cnt + 1'b1;
51    end
52
53    //Neg_Edge Detector
54    logic rx_s;
55    logic rx_d;
56    logic rx_neg;
57
58    always_ff@(posedge clk)
59    begin
60        if(~rst) begin
61            rx_s <= 1'b1;
62            rx_d <= 1'b1;
63            rx_neg <= 1'b0;
64        end
65        else begin
66            {rx_d, rx_s} <= {rx_s, rx};
67            rx_neg <= ~rx_s & rx_d;
68        end
69    end
70
71    //assign rx_neg = ~rx_s & rx_d;
72
73    //baud_cnt_compare
74    logic baud_cnt_max;
75    logic baud_cnt_half;
76    assign baud_cnt_max = (baud_cnt == 1301) ? 1 : 0;
77    assign baud_cnt_half = (baud_cnt == 650) ? 1 : 0;
78
79    //bit_shift_register
80    logic [7:0]rx_data;
81
82    always_ff@(posedge clk)
83    begin
84        if(~rst)
85            rx_data <= 0;
```

```

86         else if(bit_flag)
87             rx_data <= {rx,rx_data[7:1]};
88     end
89
90     //pkg_shift_register
91     logic [7:0] head;
92     logic [7:0] addr1;
93     logic [7:0] addr2;
94     logic [7:0] data1;
95     logic [7:0] data2;
96     logic [7:0] r_w;
97     logic [7:0] chk_sum;
98     logic [7:0] tail;
99     logic shift_pkg;
100
101     always_ff@(posedge clk)
102     begin
103         if(~rst) begin
104             head <= 0;
105             addr1 <= 0;
106             addr2 <= 0;
107             data1 <= 0;
108             data2 <= 0;
109             r_w <= 0;
110             chk_sum <= 0;
111             tail <= 0;
112         end
113         else if(shift_pkg) begin
114             case(pkg_cnt)
115                 0: head <= rx_data;
116                 1: addr1 <= rx_data;
117                 2: addr2 <= rx_data;
118                 3: data1 <= rx_data;
119                 4: data2 <= rx_data;
120                 5: r_w <= rx_data;
121                 6: tail <= rx_data;
122                 7: tail <= rx_data;
123             endcase
124         end
125     end
126     //check_cum_accumulator
127     logic add_chk_sum;
128     logic rst_chk_sum_acc;
129     logic [7:0] chk_sum_acc;
130
131     always_ff@(posedge clk)
132     begin
133         if(~rst || rst_chk_sum_acc)
134             chk_sum_acc <= 0;
135         else if(add_chk_sum)
136             chk_sum_acc <= chk_sum_acc + rx_data;
137     end
138
139     //TX_Counter
140     logic inc_tx_cnt;
141     logic rst_tx_cnt;
142
143     always_ff@(posedge clk)
144     begin
145         if(~rst || rst_tx_cnt)
146             tx_cnt <= 0;
147         else if(inc_tx_cnt)
148             tx_cnt <= tx_cnt + 1'b1;
149     end
150     //data addr
151
152     typedef enum{ START, RX_START, NUM_BITS, RECEIVE, COMPLETE, RW_REG_F, TX_ACK_1,
153 TX_REQ_1, TX_ACK_2, TX_REQ_2, TX_ACK_3, TX_REQ_3, TX_ACK_4, TX_REQ_4 } FSM_STATE;
154     FSM_STATE fsm_ns,fsm_ps;
155
156     always_ff@(posedge clk)
157     begin
158         if(~rst) begin
159             data <= 0;
160             addr <= 0;
161         end
162         else if(fsm_ps == COMPLETE && pkg_cnt == 5) begin
163             data = {data1[3:0], data2[3:0]};
164             addr = {addr1[3:0], addr2[3:0]};
165         end
166     end
167
168     //FSM
169

```

```

170
171 always_ff@(posedge clk) begin
172     if(~rst)
173         fsm_ps      <= START;
174     else
175         fsm_ps      <= fsm_ns;
176 end
177
178 always_comb
179 begin
180     fsm_ns          = fsm_ps;
181     rst_baud_cnt    = 0;
182     bit_flag        = 0;
183     rst_bit_cnt     = 0;
184     rx_finish       = 0;
185     rst_pkg_cnt     = 0;
186     add_chk_sum     = 0;
187     rst_chk_sum_acc = 0;
188     write           = 0;
189     shift_pkg       = 0;
190     inc_tx_cnt      = 0;
191     rst_tx_cnt      = 0;
192     tx_req          = 0;
193     case(fsm_ps)
194     START:
195     begin
196         rst_chk_sum_acc = 1;
197         fsm_ns = RX_START;
198     end
199     RX_START:
200     begin
201         rst_chk_sum_acc = 1;
202         if(rx_neg == 1)
203             fsm_ns = NUM_BITS;
204     end
205     NUM_BITS:
206     begin
207         rst_baud_cnt = 1;
208         if(bit_cnt>8)
209             fsm_ns = COMPLETE;
210         else
211             fsm_ns = RECEIVE;
212     end
213     RECEIVE:
214     begin
215         if(baud_cnt_half == 1)
216             bit_flag = 1;
217         else if(baud_cnt_max == 1)
218             fsm_ns = NUM_BITS;
219     end
220     COMPLETE:
221     begin
222         rst_bit_cnt = 1;
223         rx_finish = 1;
224         shift_pkg = 1;
225         if(pkg_cnt == 6) begin
226             rst_pkg_cnt = 1;
227             rst_chk_sum_acc = 1;
228             fsm_ns = RW_REG_F;
229         end
230         else if(pkg_cnt<7) begin
231             add_chk_sum = 1;
232             fsm_ns = RX_START;
233         end
234         else
235             fsm_ns = COMPLETE;
236     end
237     RW_REG_F:
238     begin
239         rst_tx_cnt = 1;
240         if(r_w[0]) begin
241             write = 1;
242             fsm_ns = RX_START;
243         end
244         else
245             fsm_ns = TX_REQ_1;
246     end
247     TX_REQ_1:
248     begin
249         tx_req = 1;
250         fsm_ns = TX_ACK_1;
251     end
252     TX_ACK_1:
253

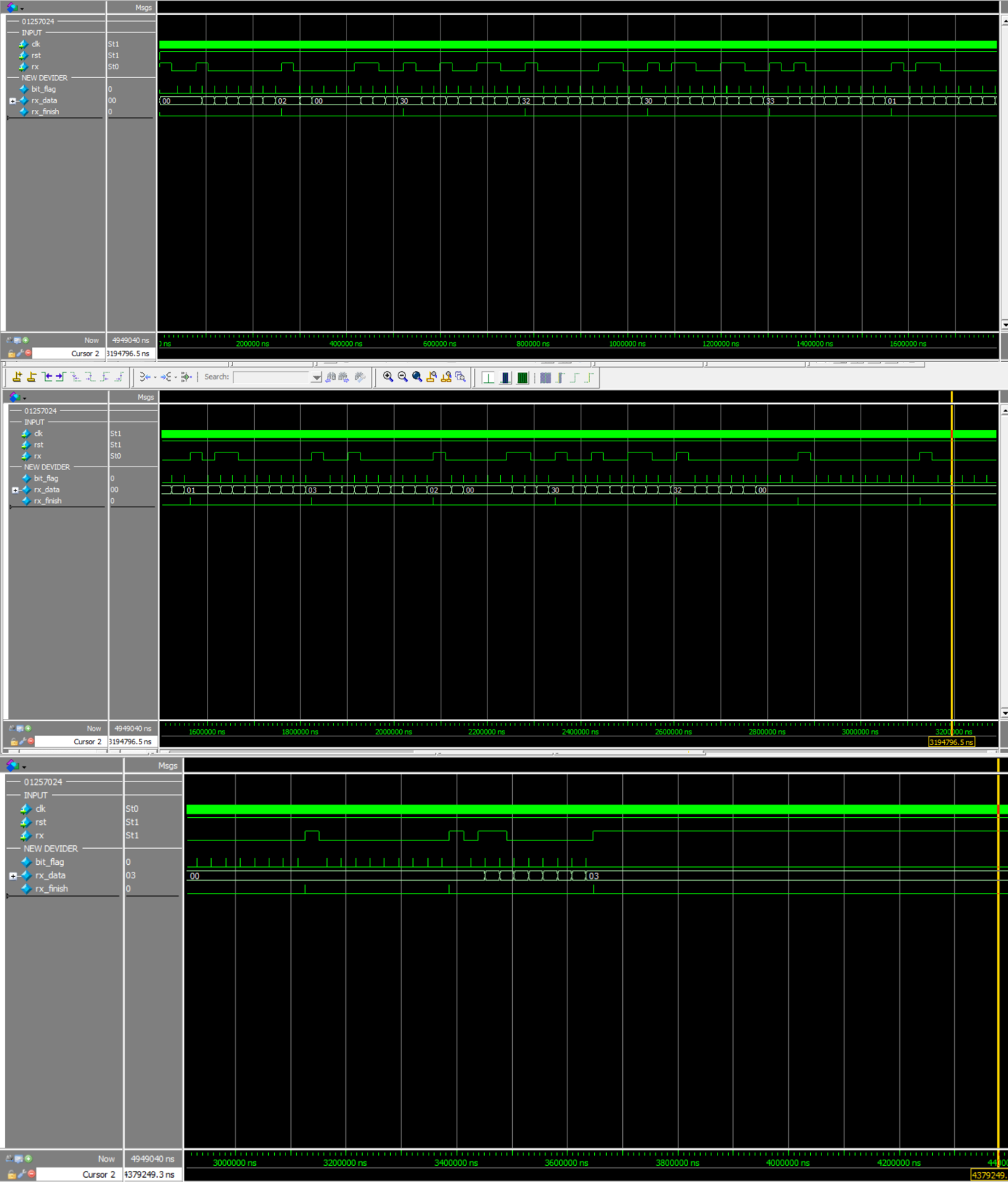
```

```

254 begin
255     if(tx_ack == 1)
256         fsm_ns = TX_REQ_2;
257     end
258 TX_REQ_2:
259 begin
260     tx_req = 1;
261     inc_tx_cnt = 1;
262     fsm_ns = TX_ACK_2;
263 end
264 TX_ACK_2:
265 begin
266     if(tx_ack == 1)
267         fsm_ns = TX_REQ_3;
268     end
269 TX_REQ_3:
270 begin
271     tx_req = 1;
272     inc_tx_cnt = 1;
273     fsm_ns = TX_ACK_3;
274 end
275 TX_ACK_3:
276 begin
277     if(tx_ack == 1)
278         fsm_ns = TX_REQ_4;
279     end
280 TX_REQ_4:
281 begin
282     tx_req = 1;
283     inc_tx_cnt = 1;
284     fsm_ns = TX_ACK_4;
285 end
286 TX_ACK_4:
287 begin
288     if(tx_ack == 1)
289         fsm_ns = RX_START;
290     end
291 endcase
292 end
293 endmodule
294

```

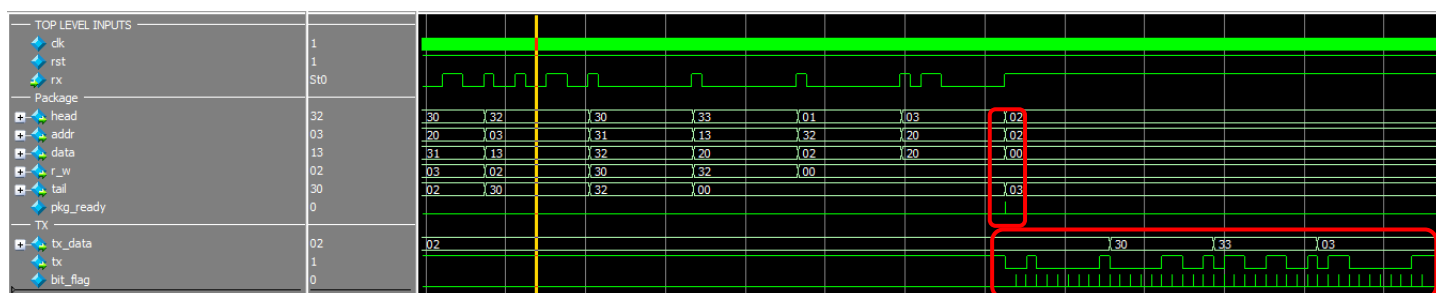
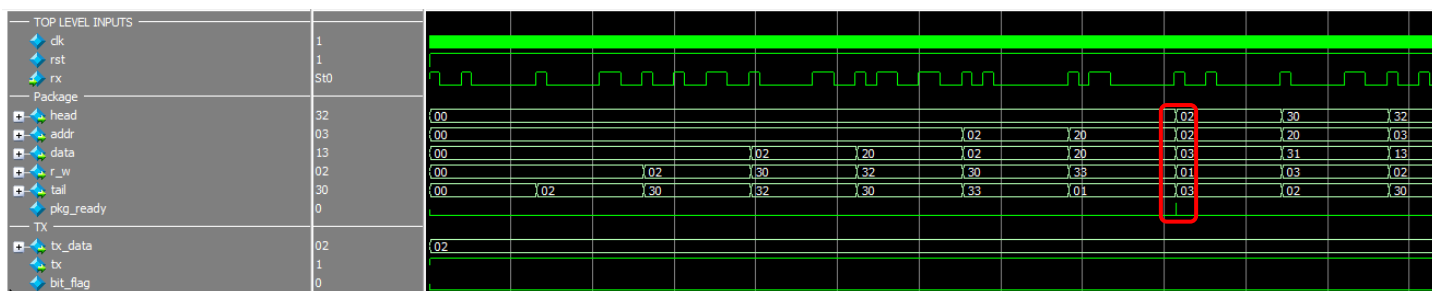
■ 模擬結果與結果說明：



2、 RS-232 Reciever (二)

■ 實驗說明：

1. 承上題，接收並解析整個 package 資料，依格式將資料分類輸出，並結合 RS-232 Transmitter 將資料依照 package 格式回傳。
2. 加分題(可做可不作)：Checksum 實作。若 Checksum 檢測為 False，則拋棄此 package 資料(不讀不寫)；否則依格式資料進行讀寫。
3. 輸入：
 - clk : 50M
 - rst
 - rx : 接收到資料(1-bit)
4. 輸出：
 - head[7:0] : 8'h02
 - addr[7:0] : 解析完的真實 address
 - data[7:0] : 解析完的真實 data
 - r_w[7:0] : 讀取(8'h00)或是寫入(8'h01)
 - tail[7:0] : 8'h03
 - pkg_ready : 標示完整接收到 1 個 package 的 trigger。
 - tx_data[7:0] : 要給 transmitter 回傳的資料。
 - tx : 傳出去的資料(1-bit)
5. 測資：
 - 8'h02, 8'h30, 8'h32, 8'h30, 8'h33, 8'h01, 8'h03
 - 8'h02, 8'h30, 8'h32, 8'h00, 8'h00, 8'h00, 8'h03



6. 加分題測資 - Checksum : testbench 請自行修改。

- 8'h02, 8'h30, 8'h32, 8'h30, 8'h33, 8'h01, **8'hc8**, 8'h03
- 8'h02, 8'h31, 8'h33, 8'h32, 8'h39, 8'h01, **8'hd3**, 8'h03 // **checksum error**
- 8'h02, 8'h31, 8'h33, 8'h39, 8'h32, 8'h01, **8'hd2**, 8'h03
- 8'h02, 8'h30, 8'h32, 8'h00, 8'h00, 8'h00, **8'h64**, 8'h03
- 8'h02, 8'h31, 8'h33, 8'h00, 8'h00, 8'h00, **8'h66**, 8'h03

■ 系統架構程式碼與程式碼說明

RS232_rx 跟上一題的一樣

RS232_tx

```
1 module rs232_tx(
2     input logic clk,
3     input logic rst,
4     input logic tx_req,
5     input logic [7:0] tx_data,
6     output logic tx,
7     output logic tx_ack
8 );
9
10
11 //baud_counter
12 logic rst_baud_cnt;
13 logic [15:0] baud_cnt;
14
15 always_ff@(posedge clk)
16 begin
17     if(~rst || rst_baud_cnt)
18         baud_cnt <= 0;
19     else
20         baud_cnt <= baud_cnt + 1'b1;
21     end
22
23 //bit_counter
24 logic [3:0] bit_cnt;
25 logic rst_bit_cnt;
26 logic bit_flag;
27
28 always_ff@(posedge clk)
29 begin
30     if(~rst || rst_bit_cnt)
31         bit_cnt <= 0;
32     else if(bit_flag)
33         bit_cnt <= bit_cnt + 1'b1;
34     end
35
36 //baud_cnt_compare
37 logic baud_cnt_max;
38 assign baud_cnt_max = (baud_cnt == 1301) ? 1 : 0;
39
40 //shift_register
41 logic [9:0] data;
42 logic load_tx_data;
43
44 always_ff@(posedge clk)
45 begin
46     if(~rst)
47         data <= 9'b11111111;
48     else if(load_tx_data)
49         data <= {1'b1, tx_data, 1'b0};
50     else if(bit_flag)
51         data <= {1'b1, data[9:1]};
52     end
53
54 //output_register
55 logic send_tx_data;
56
57 always_ff@(posedge clk)
58 begin
59     if(~rst || tx_ack)
60         tx <= 1'b1;
61     else if(send_tx_data)
62         tx <= data[0];
63     end
64
65 //FSM
66 typedef enum{ IDLE, CHK_REQ, LD_TX_D,
67     CHK_BIT_CNT, TRANS, COMP} FSM_STATE;
68 FSM_STATE fsm_ns, fsm_ps;
69
70 always_ff@(posedge clk) begin
71     if(~rst)
72         fsm_ps <= IDLE;
73     else
74         fsm_ps <= fsm_ns;
75     end
76
77 always_comb
78 begin
79     fsm_ns = fsm_ps;
80     rst_baud_cnt = 0;
81     rst_bit_cnt = 0;
82     bit_flag = 0;
83     load_tx_data = 0;
84     tx_ack = 0;
85     send_tx_data = 0;
```

```

86 case(fsm_ps)
87     IDLE:
88     begin
89         fsm_ns = CHK_REQ;
90     end
91     CHK_REQ:
92     begin
93         if(tx_req)
94             fsm_ns = LD_TX_D;
95         end
96     LD_TX_D:
97     begin
98         load_tx_data = 1;
99         fsm_ns = CHK_BIT_CNT;
100     end
101     CHK_BIT_CNT:
102     begin
103         rst_baud_cnt = 1;
104         send_tx_data = 1;
105         if(bit_cnt == 10)
106             fsm_ns = COMP;
107         else
108             fsm_ns = TRANS;
109         end
110     TRANS:
111     begin
112         send_tx_data = 1;
113         if(baud_cnt_max) begin
114             bit_flag = 1;
115             fsm_ns = CHK_BIT_CNT;
116         end
117     end
118     COMP:
119     begin
120         rst_bit_cnt = 1;
121         tx_ack = 1;
122         fsm_ns = CHK_REQ;
123     end
124 endcase
125 end
126
127 L
128 endmodule
129

```

RS232 (TOP LEVEL)

```
1 module RS232(  
2     input logic clk,  
3     input logic rst,  
4     input logic r_LPF_threshold,  
5     input logic rx,  
6     output logic tx  
7 );  
8 //Low_Pass_Filter  
9 logic rx_filter;  
10 Low_Pass_Filter Low_Pass_Filter_1(  
11     .clk      (clk),  
12     .reset    (rst),  
13     .r_LPF_threshold (r_LPF_threshold ),  
14     .signal   (rx),  
15     .sig_filter (rx_filter)  
16 );  
17  
18 //RS232_RX  
19 logic tx_ack;  
20 logic tx_req;  
21 logic [7:0]addr;  
22 logic [7:0]data;  
23 logic write;  
24 logic [1:0]tx_cnt;  
25 rs232_rx rs232_rx_1(  
26     .clk      (clk),  
27     .rst      (rst),  
28     .tx_ack   (tx_ack),  
29     .rx       (rx_filter),  
30     .addr     (addr),  
31     .data     (data),  
32     .write    (write),  
33     .tx_req   (tx_req),  
34     .tx_cnt   (tx_cnt)  
35 );  
36  
37 //register file  
38 logic [7:0] reg_file [255:0];  
39 logic [7:0] data_r;  
40 always_ff@(posedge clk)  
41 begin  
42     if(write)  
43         reg_file[addr] <= data;  
44 end  
45  
46 assign data_r = reg_file[addr];  
47  
48 //Multiplexer  
49 logic [7:0] tx_data;  
50 always_comb  
51 begin  
52     case(tx_cnt)  
53         0: tx_data = 8'h02;  
54         1: tx_data = {4'h3, data_r[7:4]};  
55         2: tx_data = {4'h3, data_r[3:0]};  
56         3: tx_data = 8'h03;  
57     endcase  
58 end  
59  
60 //RS232_TX  
61 rs232_tx rs232_tx_1(  
62     .clk      (clk),  
63     .rst      (rst),  
64     .tx_ack   (tx_ack),  
65     .tx_req   (tx_req),  
66     .tx_data  (tx_data),  
67     .tx       (tx)  
68 );  
69  
70 endmodule  
71
```

加分題

加分題只有 RS232_rx 有變

所以只有截圖 RS232_rx 的圖

```
1 module rs232_rx(  
2     input logic clk,  
3     input logic rst,  
4     input logic tx_ack,  
5     input logic rx,  
6     output logic [7:0]addr,  
7     output logic [7:0]data,  
8     output logic write,  
9     output logic tx_req,  
10    output logic [1:0]tx_cnt  
11 );  
12  
13  
14 //bit_counter  
15 logic rst_bit_cnt;  
16 logic bit_flag;  
17 logic [5:0]bit_cnt;  
18  
19 always_ff@(posedge clk)  
20 begin  
21     if(~rst || rst_bit_cnt)  
22         bit_cnt <= 0;  
23     else if(bit_flag)  
24         bit_cnt <= bit_cnt + 1'b1;  
25 end  
26  
27  
28 //baud_counter  
29 logic rst_baud_cnt;  
30 logic [15:0]baud_cnt;  
31  
32 always_ff@(posedge clk)  
33 begin  
34     if(~rst || rst_baud_cnt)  
35         baud_cnt <= 0;  
36     else  
37         baud_cnt <= baud_cnt + 1'b1;  
38 end  
39  
40 //pkg_counter  
41 logic [2:0]pkg_cnt;  
42 logic rst_pkg_cnt;  
43 logic rx_finish;  
44  
45 always_ff@(posedge clk)  
46 begin  
47     if(~rst || rst_pkg_cnt)  
48         pkg_cnt <= 0;  
49     else if(rx_finish)  
50         pkg_cnt <= pkg_cnt + 1'b1;  
51 end  
52  
53 //Neg_Edge Detector  
54 logic rx_s;  
55 logic rx_d;  
56 logic rx_neg;  
57  
58 always_ff@(posedge clk)  
59 begin  
60     if(~rst) begin  
61         rx_s <= 1'b1;  
62         rx_d <= 1'b1;  
63         rx_neg <= 1'b0;  
64     end  
65     else begin  
66         {rx_d, rx_s} <= {rx_s, rx};  
67         rx_neg <= ~rx_s & rx_d;  
68     end  
69 end  
70  
71 //assign rx_neg = ~rx_s & rx_d;  
72  
73 //baud_cnt_compare  
74 logic baud_cnt_max;  
75 logic baud_cnt_half;  
76 assign baud_cnt_max = (baud_cnt == 1301) ? 1 : 0;  
77 assign baud_cnt_half = (baud_cnt == 650) ? 1 : 0;  
78  
79 //bit_shift_register  
80 logic [7:0]rx_data;  
81  
82 always_ff@(posedge clk)  
83 begin  
84     if(~rst)  
85         rx_data <= 0;
```

```

86         else if(bit_flag)
87             rx_data <= {rx,rx_data[7:1]};
88     end
89
90     //pkg_shift_register
91     logic [7:0] head;
92     logic [7:0] addr1;
93     logic [7:0] addr2;
94     logic [7:0] data1;
95     logic [7:0] data2;
96     logic [7:0] r_w;
97     logic [7:0] chk_sum;
98     logic [7:0] tail;
99     logic shift_pkg;
100
101     always_ff@(posedge clk)
102     begin
103         if(~rst) begin
104             head <= 0;
105             addr1 <= 0;
106             addr2 <= 0;
107             data1 <= 0;
108             data2 <= 0;
109             r_w <= 0;
110             chk_sum <= 0;
111             tail <= 0;
112         end
113         else if(shift_pkg) begin
114             case(pkg_cnt)
115                 0: head <= rx_data;
116                 1: addr1 <= rx_data;
117                 2: addr2 <= rx_data;
118                 3: data1 <= rx_data;
119                 4: data2 <= rx_data;
120                 5: r_w <= rx_data;
121                 6: chk_sum <= rx_data;
122                 7: tail <= rx_data;
123             endcase
124         end
125     end
126     //check_cum_accumulator
127     logic add_chk_sum;
128     logic rst_chk_sum_acc;
129     logic [7:0] chk_sum_acc;
130
131     always_ff@(posedge clk)
132     begin
133         if(~rst || rst_chk_sum_acc)
134             chk_sum_acc <= 0;
135         else if(add_chk_sum)
136             chk_sum_acc <= chk_sum_acc + rx_data;
137     end
138
139     //TX_Counter
140     logic inc_tx_cnt;
141     logic rst_tx_cnt;
142
143     always_ff@(posedge clk)
144     begin
145         if(~rst || rst_tx_cnt)
146             tx_cnt <= 0;
147         else if(inc_tx_cnt)
148             tx_cnt <= tx_cnt + 1'b1;
149     end
150     //data addr
151
152     typedef enum{ START, RX_START, NUM_BITS, RECEIVE, COMPLETE, RW_REG_F, TX_ACK_1,
153 TX_REQ_1, TX_ACK_2, TX_REQ_2, TX_ACK_3, TX_REQ_3, TX_ACK_4, TX_REQ_4 } FSM_STATE;
154     FSM_STATE fsm_ns,fsm_ps;
155
156     always_ff@(posedge clk)
157     begin
158         if(~rst) begin
159             data <= 0;
160             addr <= 0;
161         end
162         else if(fsm_ps == COMPLETE && pkg_cnt == 5) begin
163             data = {data1[3:0], data2[3:0]};
164             addr = {addr1[3:0], addr2[3:0]};
165         end
166     end
167
168     //FSM
169

```

```

170
171 always_ff@(posedge clk) begin
172     if(~rst)
173         fsm_ps      <= START;
174     else
175         fsm_ps      <= fsm_ns;
176 end
177
178 always_comb
179 begin
180     fsm_ns          = fsm_ps;
181     rst_baud_cnt    = 0;
182     bit_flag        = 0;
183     rst_bit_cnt     = 0;
184     rx_finish       = 0;
185     rst_pkg_cnt     = 0;
186     add_chk_sum     = 0;
187     rst_chk_sum_acc = 0;
188     write           = 0;
189     shift_pkg       = 0;
190     inc_tx_cnt      = 0;
191     rst_tx_cnt      = 0;
192     tx_req          = 0;
193     case(fsm_ps)
194     START:
195     begin
196         rst_chk_sum_acc = 1;
197         fsm_ns = RX_START;
198     end
199     RX_START:
200     begin
201         if(rx_neg == 1)
202             fsm_ns = NUM_BITS;
203     end
204     NUM_BITS:
205     begin
206         rst_baud_cnt = 1;
207         if(bit_cnt>8)
208             fsm_ns = COMPLETE;
209         else
210             fsm_ns = RECEIVE;
211     end
212     RECEIVE:
213     begin
214         if(baud_cnt_half == 1)
215             bit_flag = 1;
216         else if(baud_cnt_max == 1)
217             fsm_ns = NUM_BITS;
218     end
219     COMPLETE:
220     begin
221         rst_bit_cnt = 1;
222         rx_finish = 1;
223         shift_pkg = 1;
224         if(pkg_cnt == 7) begin
225             if(chk_sum == chk_sum_acc)
226                 fsm_ns = RW_REG_F;
227             else
228                 fsm_ns = RX_START;
229             rst_pkg_cnt = 1;
230             rst_chk_sum_acc = 1;
231         end
232         else if(pkg_cnt<7) begin
233             if(pkg_cnt<6)
234                 add_chk_sum = 1;
235             fsm_ns = RX_START;
236         end
237         else
238             fsm_ns = COMPLETE;
239     end
240     RW_REG_F:
241     begin
242         rst_tx_cnt = 1;
243         if(r_w[0]) begin
244             write = 1;
245             fsm_ns = RX_START;
246         end
247         else
248             fsm_ns = TX_REQ_1;
249     end
250     TX_REQ_1:
251
252
253

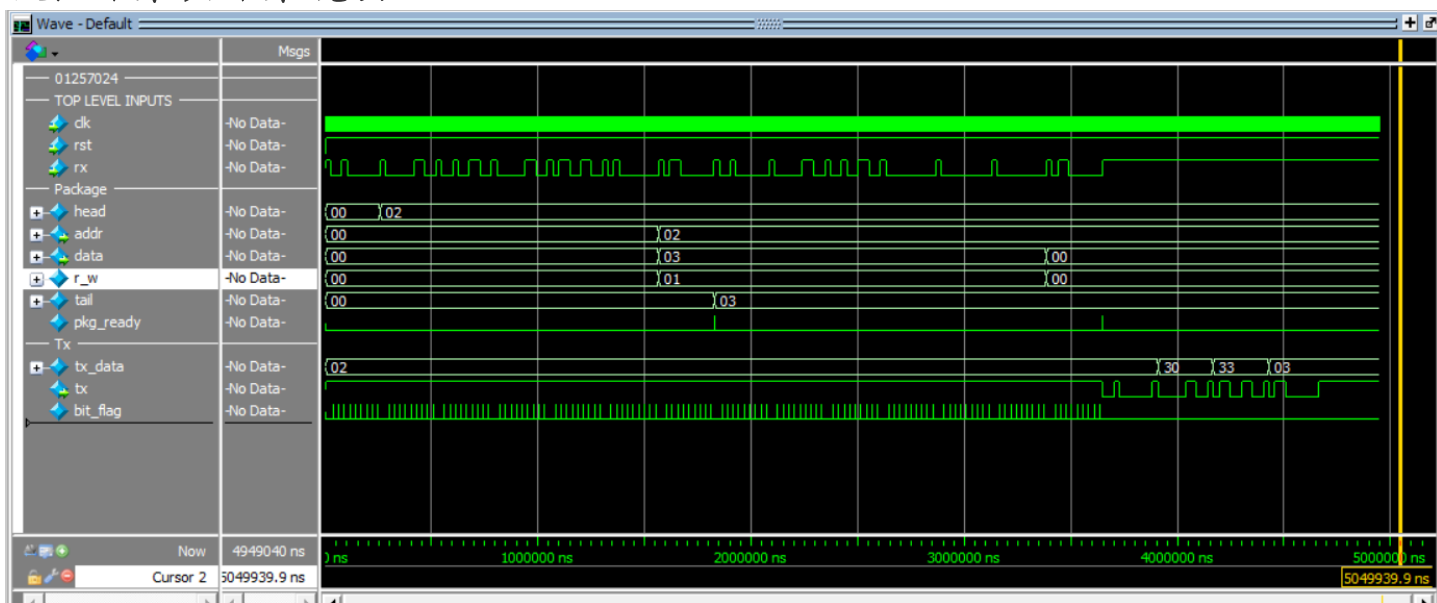
```

```

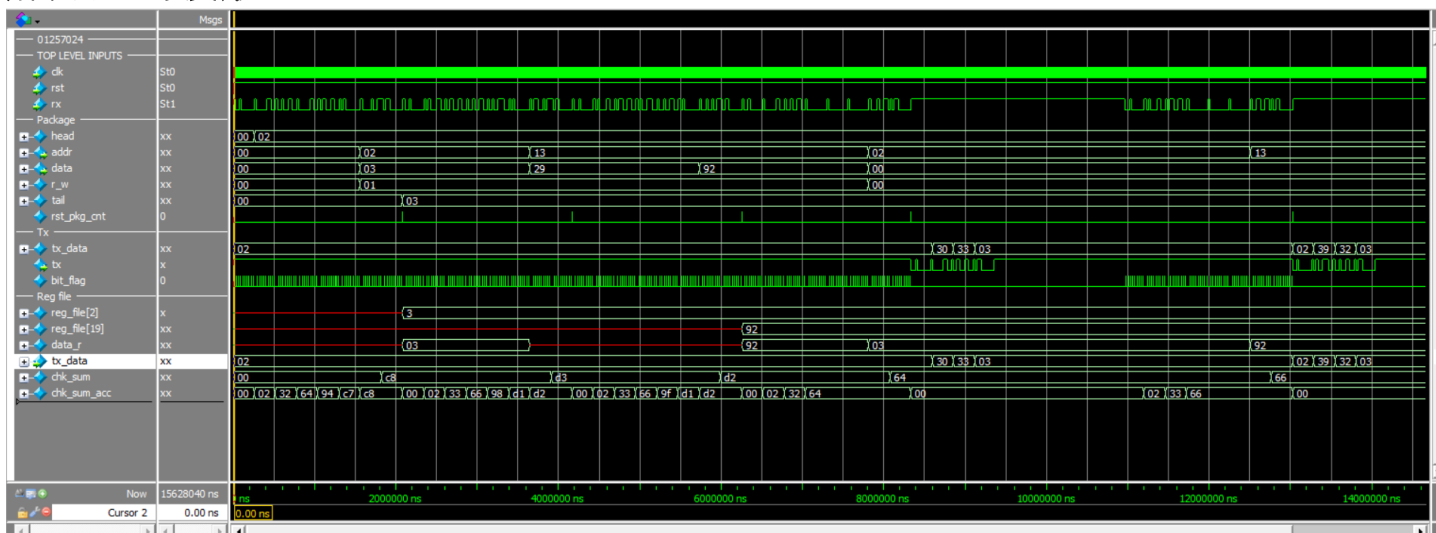
254 begin
255     tx_req = 1;
256     fsm_ns = TX_ACK_1;
257 end
258 TX_ACK_1:
259 begin
260     if(tx_ack == 1)
261         fsm_ns = TX_REQ_2;
262     end
263 TX_REQ_2:
264 begin
265     tx_req = 1;
266     inc_tx_cnt = 1;
267     fsm_ns = TX_ACK_2;
268 end
269 TX_ACK_2:
270 begin
271     if(tx_ack == 1)
272         fsm_ns = TX_REQ_3;
273     end
274 TX_REQ_3:
275 begin
276     tx_req = 1;
277     inc_tx_cnt = 1;
278     fsm_ns = TX_ACK_3;
279 end
280 TX_ACK_3:
281 begin
282     if(tx_ack == 1)
283         fsm_ns = TX_REQ_4;
284     end
285 TX_REQ_4:
286 begin
287     tx_req = 1;
288     inc_tx_cnt = 1;
289     fsm_ns = TX_ACK_4;
290 end
291 TX_ACK_4:
292 begin
293     if(tx_ack == 1)
294         fsm_ns = RX_START;
295     end
296 endcase
297 end
298
299 endmodule
300

```

■ 模擬結果與結果說明：



加分題的波形



■ 結論與心得：

這次還有架構圖可以看，寫起來比較有系統地在寫，好寫很多。